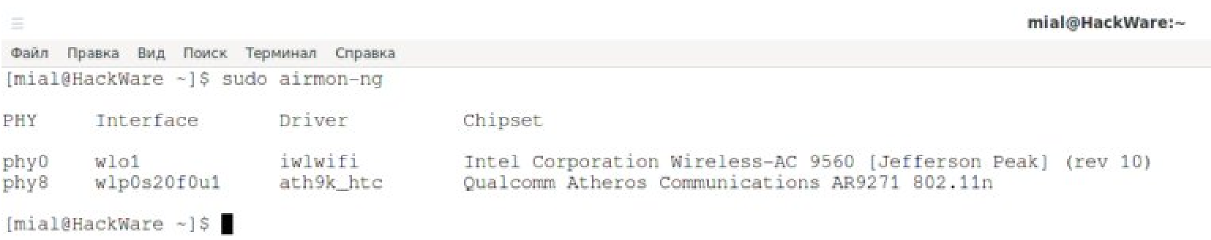
Virtual Wi-Fi interfaces for simultaneous use of a Wi-Fi adapter in different modes

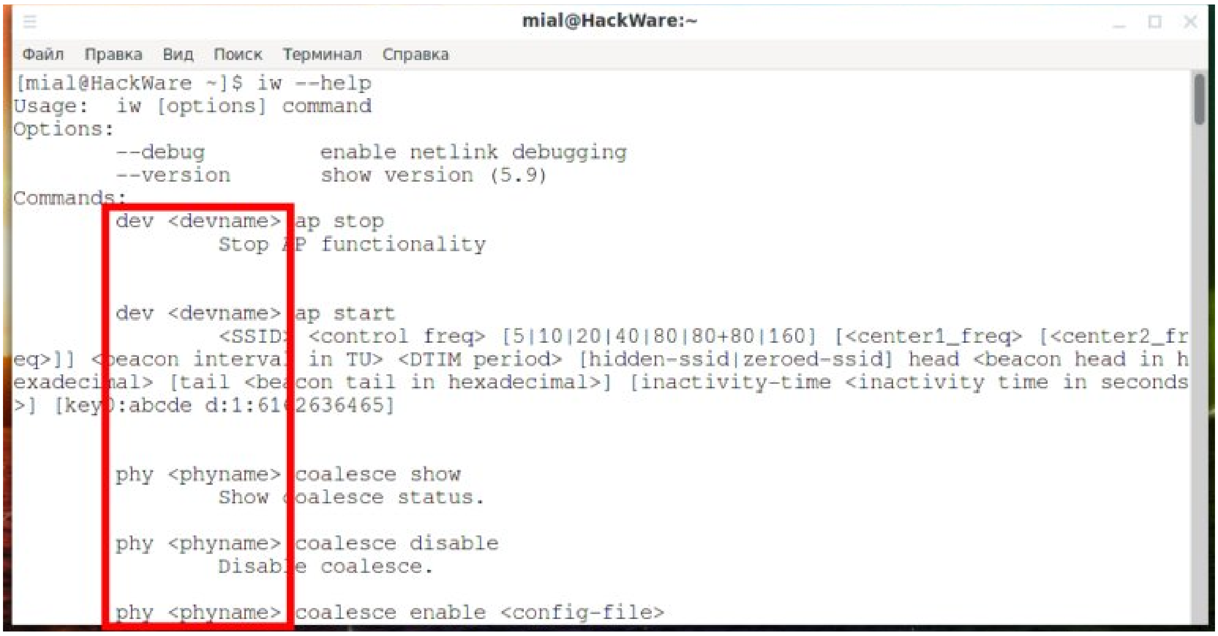
Table of contents

What is dev and phy for iw command

The Airmon-ng program for wireless adapters shows information such as **PHY** and **Interface**.



The **iw** program allows you to control the network adapter by referring to it either as a **phy** device or as a **dev** interface. In this case, the set of commands is different.



A **PHY**, an abbreviation for "physical layer", is an electronic circuit, usually implemented as an integrated circuit, required to implement physical layer functions of the OSI model in a network interface controller.

A PHY connects a link layer device (often called MAC as an acronym for medium access control) to a physical medium such as an optical fiber or copper cable. A PHY device typically includes both Physical Coding Sublayer (PCS) and Physical Medium Dependent (PMD) layer functionality.

That is, in Linux, PHY is the number assigned to the device by the PHY driver/framework for accessing it and configuring it at the data link layer.

As for **dev** (device name, interface name), this is the conventional name assigned to the device by the system. There are different naming schemes, so different Linux distributions have different names for network interfaces. If desired, the name of the network interface can be changed.

Simultaneous use of a Wi-Fi adapter in different modes

It would seem, so what? Through **phy**, you can manage settings, the meaning of which is not clear without reading special technical documentation. Using the **dev** interface name, you can control more familiar settings (switch to monitor mode, switch to a specific channel, and so on).

The point is that for wireless adapters, a network interface is not just a name that identifies a device. There are different types of interfaces and interfaces must be added to access some of the capabilities of the wireless adapters. Moreover, one Wi-Fi adapter can have several network interfaces at once that perform different functions, for example, a combination of interfaces in managed mode and in monitor mode, or a combination of interfaces in managed mode and Access Point mode, while the wireless adapter will simultaneously perform two functions at once! Moreover, different interfaces of one Wi-Fi adapter can be assigned different MAC addresses.

See also:

- How to find the MAC address and How to find the manufacturer by MAC address
- How to change MAC address in Linux, how to enable and disable automatic MAC change (spoofing) in Linux

Run the following command for a complete listing of the capabilities of all your wireless interfaces:

```
iw list
```

Look for the following lines:

- **software interface modes (can always be added)**
- **valid interface combinations** – that is, possible combinations of interfaces

Example for the first Wi-Fi adapter:

```
software interface modes (can always be added):
		 * AP/VLAN
		 * monitor
valid interface combinations:
		 * #{ managed, P2P-client } <= 2, #{ AP, mesh point, P2P-GO } <= 2,
		   total <= 2, #channels <= 1
```

That is, instead of setting the interface from managed mode to monitor mode or launching an Access Point on it, you can add a virtual interface in monitor mode or in AP mode to the existing one.

Also, this wireless adapter can have up to two network interfaces in managed, P2P-client modes, up to two AP, mesh point, P2P-GO interfaces, but in any case, the total number of interfaces cannot be more than two, while all of them have there must be one channel (#channels <= 1).

Next example:

```
software interface modes (can always be added):
		 * AP/VLAN
		 * monitor
valid interface combinations:
		 * #{ managed } <= 1, #{ AP, P2P-client, P2P-GO } <= 1, #{ P2P-device } <= 1,
		   total <= 3, #channels <= 2
```

Again, you can add an interface in monitor mode, there can be only one interface in controlled mode, there can be up to three interfaces in total and they can use up to two different channels (#channels <= 2).

Another example:

```
software interface modes (can always be added):
     * AP/VLAN
     * monitor
valid interface combinations:
     * #{ AP, mesh point } <= 4,
       total <= 4, #channels <= 1
```

Up to four AP interfaces can be added.

And another example:

```
software interface modes (can always be added):
     * AP/VLAN
     * monitor
valid interface combinations:
     * #{ AP, mesh point } <= 8,
       total <= 8, #channels <= 1
```

Up to 8 AP interfaces can be made, but all of them can use one channel.

Adding an interface in monitor mode

My preferred way to get monitor mode is to move the existing wireless interface from managed mode to monitor mode:

```
sudo ip link set <INTERFACE> down
sudo iw <INTERFACE> set monitor control
sudo ip link set <INTERFACE> up
```

As a result, the Wi-Fi card, while it is in monitor mode, loses its ability to connect to the Access Points, that is, there is no Internet connection.

See also: Linux Wi-Fi Cheat Sheet: Tips and Troubleshooting

This problem can be solved by adding a new interface.

Adding an interface in monitor mode is done as follows:

```
sudo iw INTERFACE interface add NEW_INTERFACE type monitor
```

For INTERFACE, specify the name of an existing interface, and for NEW_INTERFACE, you can specify an arbitrary name.

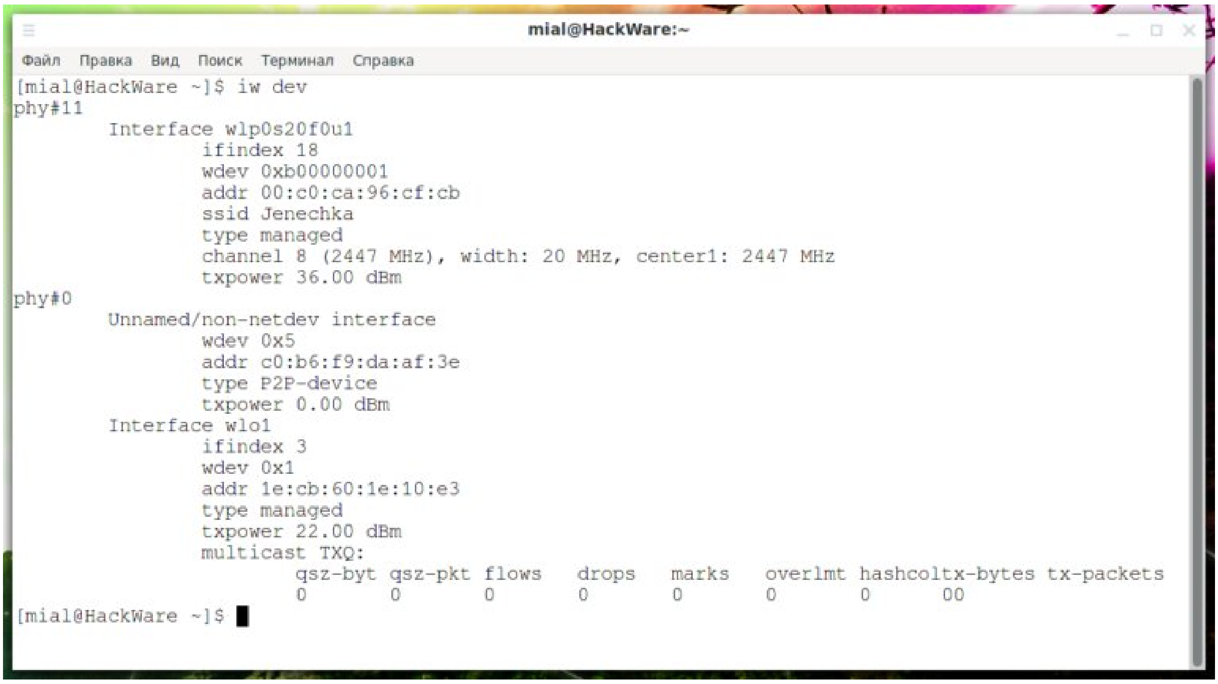
Optionally, you can assign an arbitrary MAC address to the new interface:

```
sudo iw INTERFACE interface add NEW_INTERFACE type monitor addr MAC_ADDRESS
```

Let's see what wireless interfaces are available in the system:

```
iw dev
```

We see that the wlp0s20f0u1 Wi-Fi card is connected to a network named Jenechka.

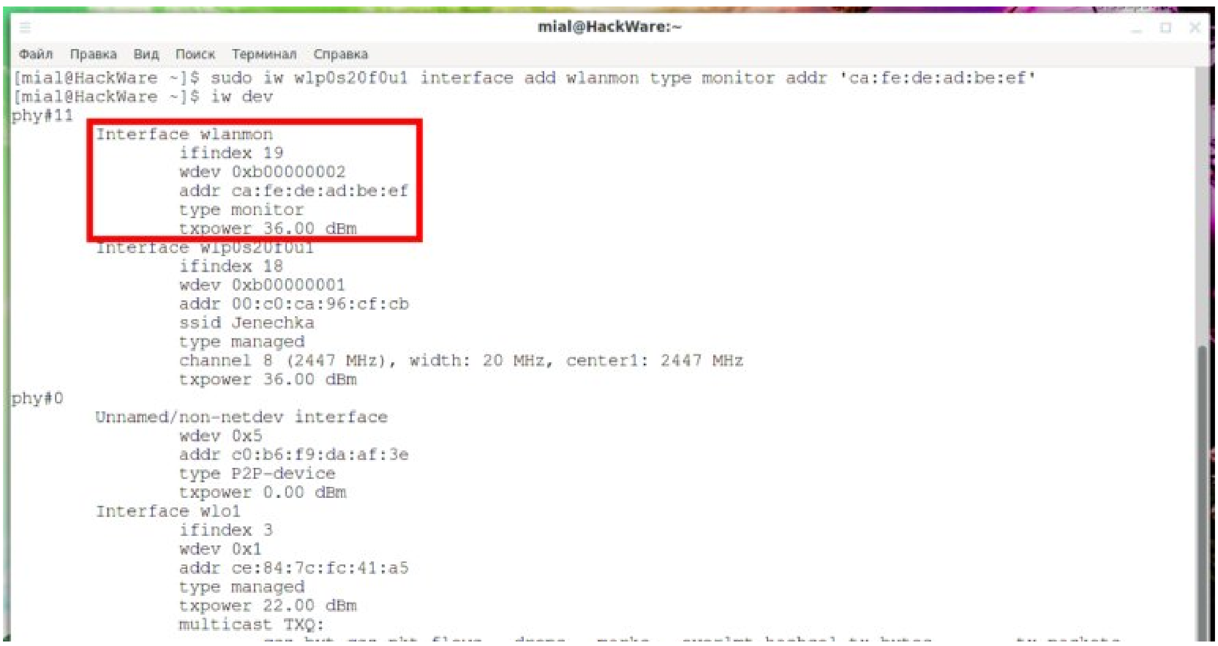


Let’s add an interface named wlanmon in monitor mode to wlp0s20f0u1 and set the MAC address on it:

```
sudo iw wlp0s20f0u1 interface add wlanmon type monitor addr 'ca:fe:de:ad:be:ef'
```

Let's take a look at our interfaces again:

```
iw dev
```



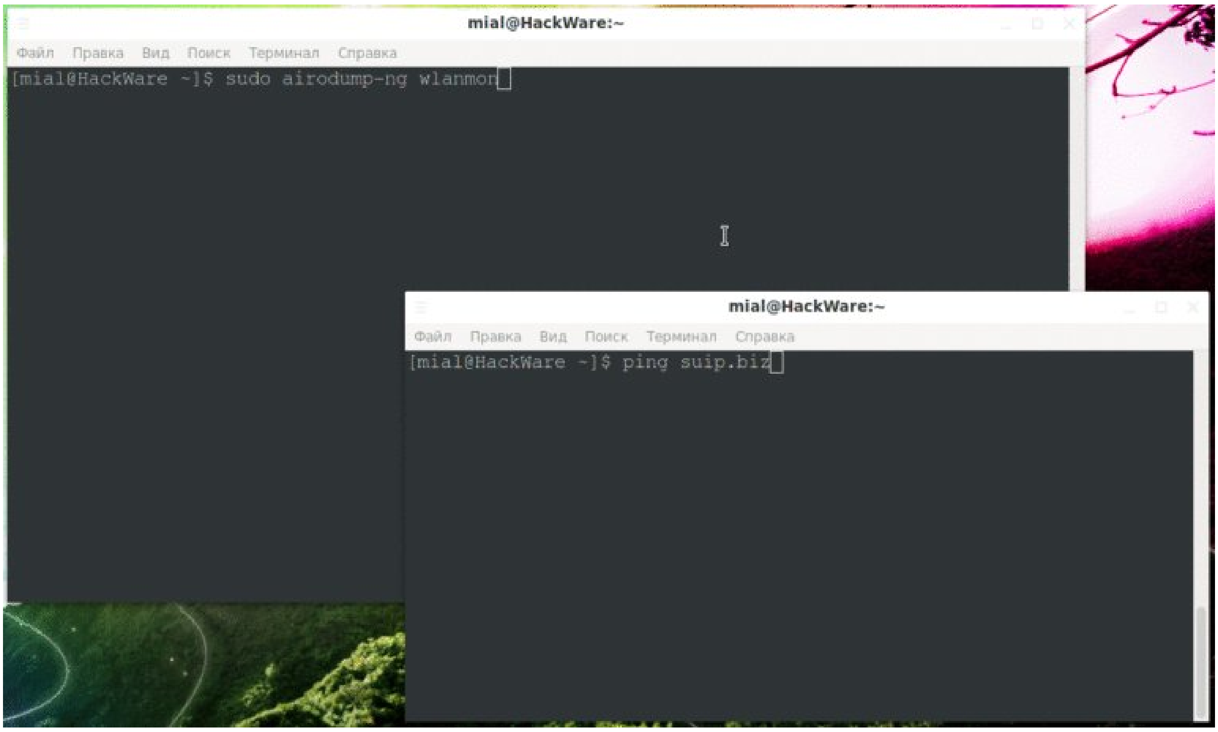
Please note that there is a new **wlanmon** interface.

Let's run airodump-ng using the newly created virtual interface:

```
sudo airodump-ng wlanmon
```

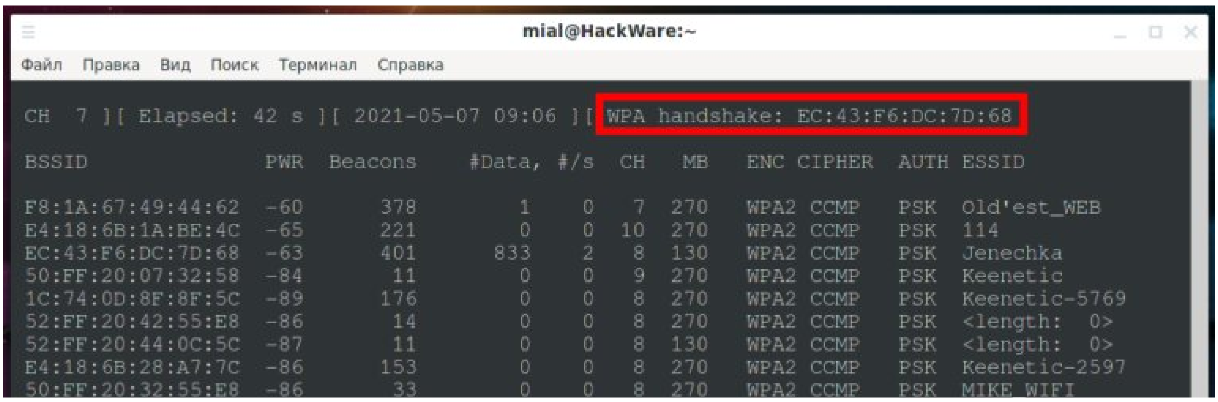
And in another window, let's start a ping to make sure that the network connection is still available:

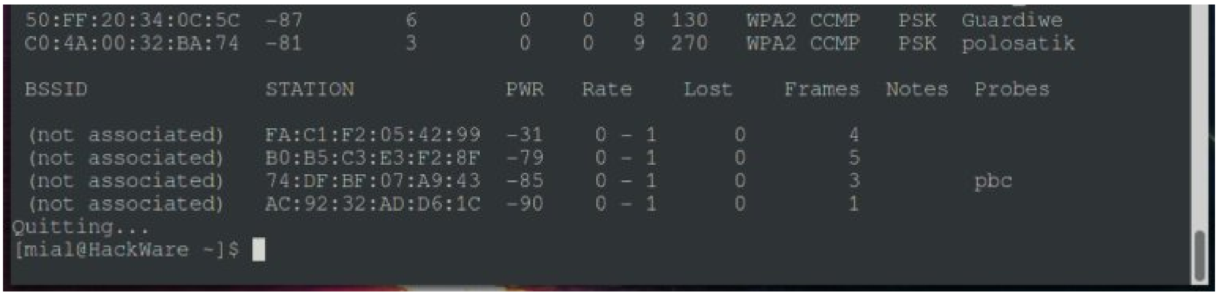
```
ping suip.biz
```



Please note that in spite of the fact that the Wi-Fi adapter needs to be on the same channel to communicate with the AP, nevertheless, the interface in the monitor mode switches over the channels.

Moreover, after stopping the screen recording, **airodump-ng** even managed to catch a handshake.




```
50:FF:20:34:0C:5C  -87        6          0    0    8  130   WPA2 CCMP   PSK  Guardiwe
C0:4A:00:32:BA:74  -81        3          0    0    9  270   WPA2 CCMP   PSK  polosatik

BSSID              STATION            PWR   Rate    Lost    Frames  Notes  Probes

(not associated)   FA:C1:F2:05:42:99  -31    0 - 1      0        4
(not associated)   B0:B5:C3:E3:F2:8F  -79    0 - 1      0        5
(not associated)   74:DF:BF:07:A9:43  -85    0 - 1      0        3          pbc
(not associated)   AC:92:32:AD:D6:1C  -90    0 - 1      0        1
Quitting...
[mial@HackWare ~]$
```

I have not noticed that the virtual interface in monitor mode, if used to capture data, interferes with the Internet connection. But if you try to send something from it (for example, to deauthenticate wireless clients), then the Internet connection is terminated. Most likely, this is due to a change in the channel used.

How to remove a virtual interface

To remove the virtual interface, use the command:

```
sudo iw INTERFACE del
```

For example:

```
sudo iw wlanmon del
```

Types of virtual interfaces

Above were shown examples of the command in which we specified the name of the network interface:

```
sudo iw dev DEV-NAME interface add NAME type NAME [mesh_id ] [4addr on|off] [flags *] [addr ]
```

You can also add a virtual interface named PHY:

```
sudo iw phy PHY-NAME interface add NAME type TYPE [mesh_id ] [4addr on|off] [flags *] [addr ]
```

This can probably be useful if you have removed all existing interfaces (if possible).

Available interface types:

- **managed**
- **ibss**
- **monitor**
- **mesh**
- **wds**

Flags are used only for monitor interfaces, valid flags are:

```
none:     no special flags
fcsfail:  show frames with FCS errors
control:  show control frames
otherbss: show frames from other BSSes
cook:     use cooked mode
active:   use active mode (ACK incoming unicast packets)
mumimo-groupid <GROUP_ID>: use MUMIMO according to a group id
mumimo-follow-mac <MAC_ADDRESS>: use MUMIMO according to a MAC address
```

The mesh_id is used only for mesh mode.

How to add an AP interface with iw

You may have noticed that there is no **AP** in the above list of valid interface types. This AP type interface will be created automatically when the Access Point is created.

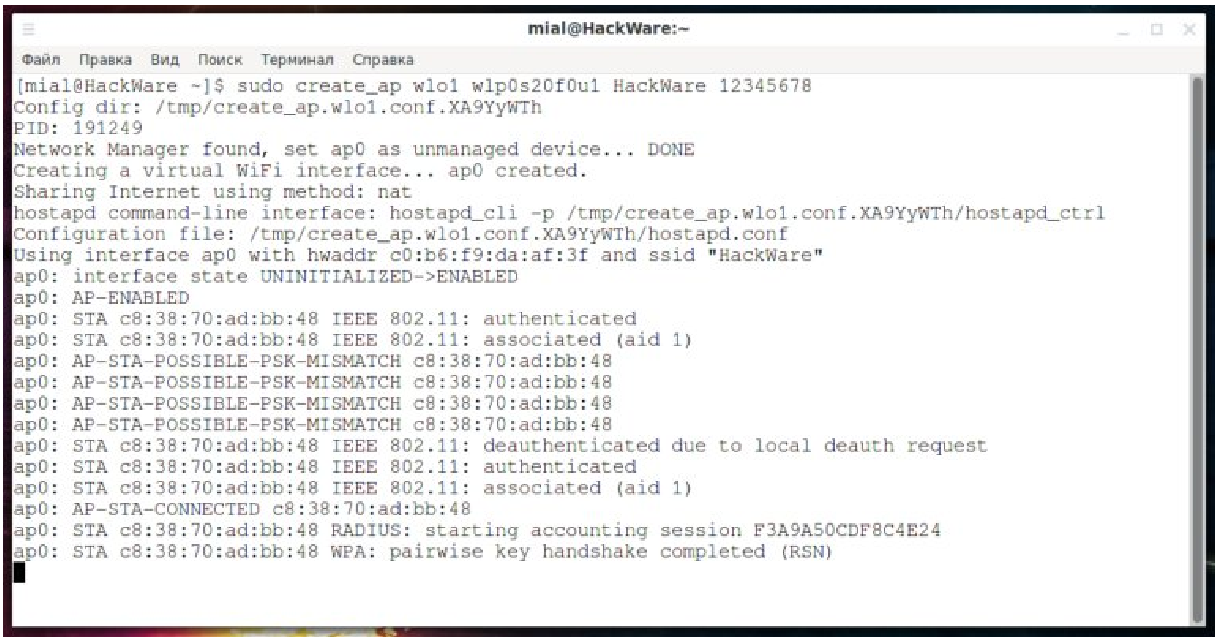
For simplicity, we will use the create_ap program (the link shows how to install it).

Usage:

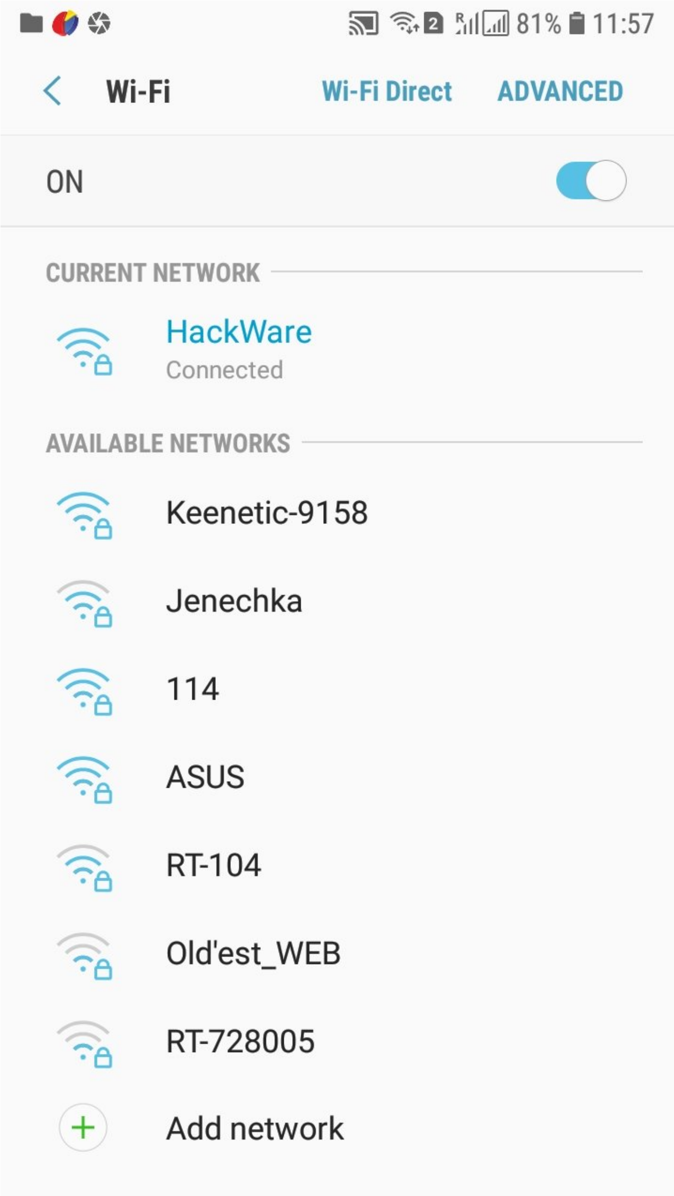
```
sudo create_ap [options] AP-INTERFACE INTERFACE-WITH-INTERNET [AP-NAME [PASSWORD]]
```

For example, on the **wlo1** interface, I want to create an AP with the name **HackWare** and the password **12345678**, the **wlp0s20f0u1** interface should be used as the Internet source, then the command is as follows:

```
sudo create_ap wlo1 wlp0s20f0u1 HackWare 12345678
```

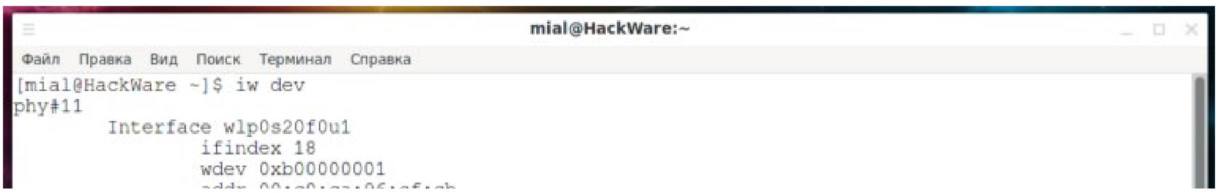


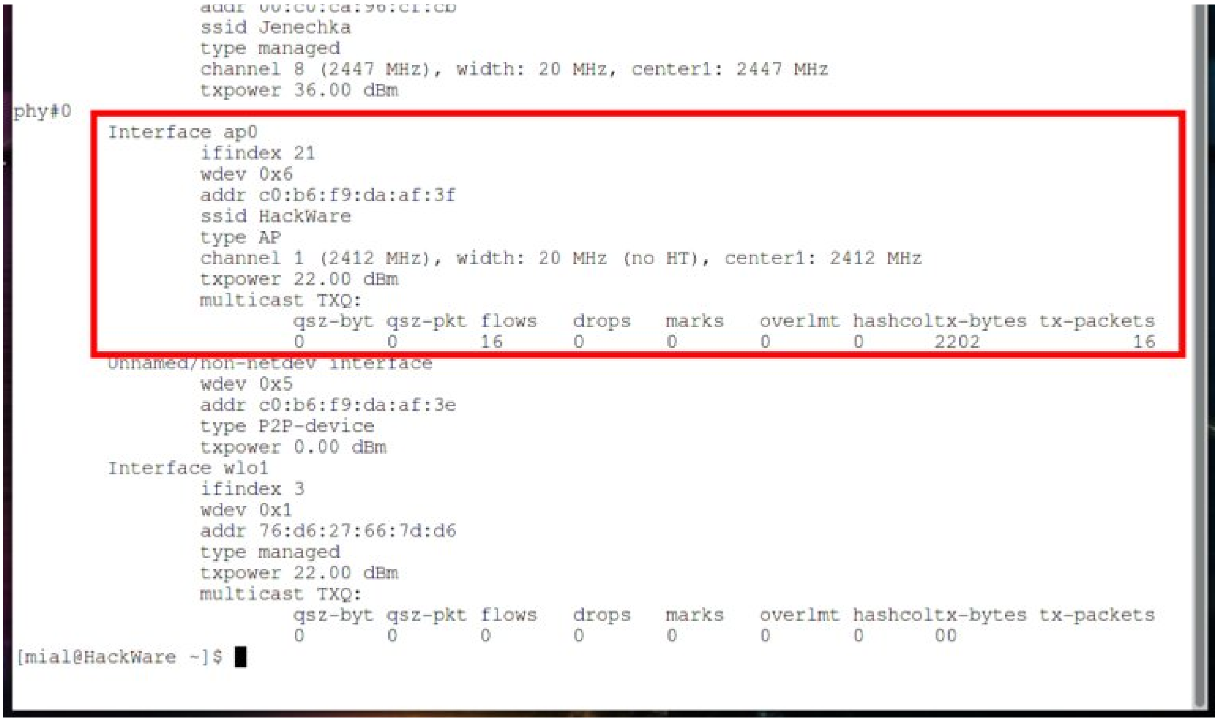
I can connect to this AP and use the Internet:



On the computer, a new interface of the AP type was created with the name ap0:

```
Interface ap0
    ifindex 21
    wdev 0x6
    addr c0:b6:f9:da:af:3f
    ssid HackWare
    type AP
    channel 1 (2412 MHz), width: 20 MHz (no HT), center1: 2412 MHz
  txpower 22.00 dBm
    multicast TXQ:
```





How to use one Wi-Fi card as a client and an Access Point

The method shown above assumes that you have two network interfaces – and one of them must be a Wi-Fi adapter, and the second can be either a Wi-Fi adapter or a wired connection.

Suppose I do not have a wired connection and I have only one adapter, can one Wi-Fi card be simultaneously used to connect to an Access Point and as an Access Point? Yes, it can. One network interface can simultaneously act as a client and as an AP, that is, it can act as a signal amplifier (wireless repeater).

Let's start by stopping NetworkManager, otherwise in my case I kept getting the error “RTNETLINK answers: Device or resource busy”:

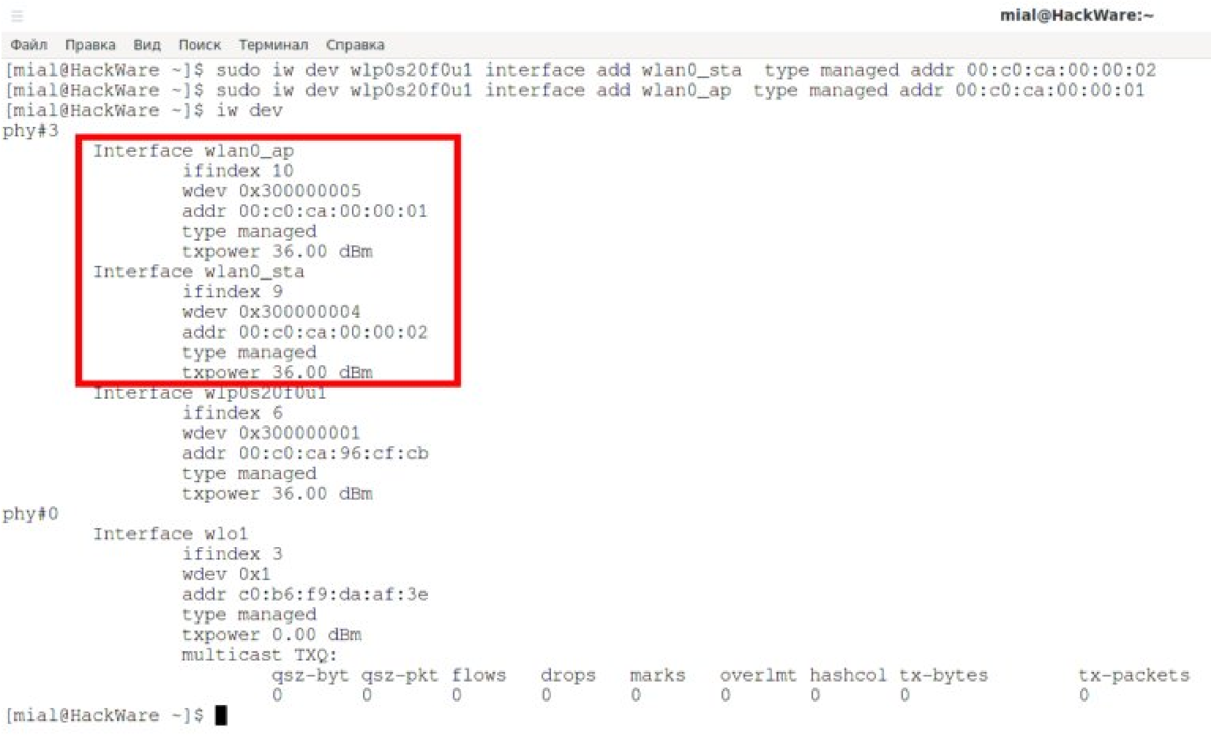
```
sudo systemctl stop NetworkManager
sudo airmon-ng check kill
```

If you have NetworkManager removed from startup, then you need to start by activating the wireless interface:

```
sudo ip link set INTERFACE up
```

Now we need to create two virtual interfaces in managed mode with different MAC addresses:

```
sudo iw dev wlp0s20f0u1 interface add wlan0_sta type managed addr 00:c0:ca:00:00:02
sudo iw dev wlp0s20f0u1 interface add wlan0_ap type managed addr 00:c0:ca:00:00:01
```



One of these interfaces will act as a station, and the other will act as an AP.

Since we disabled NetworkManager, now we need to connect to the remote AP manually.

We create a configuration file:

```
wpa_passphrase Jenechka PASSWORD > wpa_Jenechka.conf
```

We make the connection:

```
sudo wpa_supplicant -i wlan0_sta -c wpa_Jenechka.conf
```

If you want the **wpa_supplicant** process to go to the background, add the **-B** option:

```
sudo wpa_supplicant -B -i wlan0_sta -c wpa_Jenechka.conf
```

Connection takes some time:

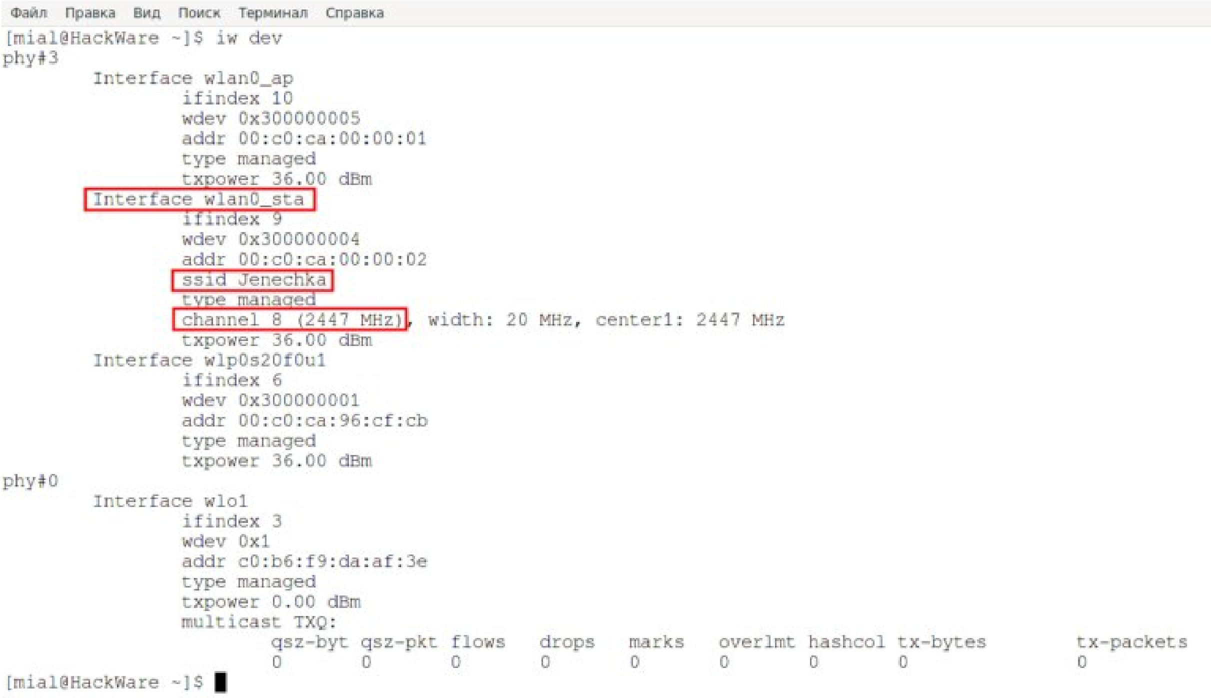
```
# sleep 5 # if run in a script, then in this place you need to give time to connect
```

We start the DHCP service to automatically obtain an IP address:

```
sudo dhclient wlan0_sta
```

We look at the state of the interfaces:

```
iw dev
```



We see that the **wlan0_sta** interface is connected to the AP – this is what we need.

Very important – look at which channel the AP and the client are working on. This channel must be specified in the next command, otherwise nothing will work. In this case, they work on channel 8.

Finally, launch an AP named **HackWare** and password **12345678** on channel **8**:

```
sudo create_ap -c 8 wlan0_ap wlan0_sta HackWare 12345678
```

Everything worked out, and our Wi-Fi signal booster even has a connected client:

Commands for displaying the names of network interfaces

A small instruction manual for obtaining information about the network interfaces available in the system.

Show only wireless interfaces:

```
iw dev
```

Output of wireless interfaces along with their PHY, driver and chipset information:

```
sudo airmon-ng
```

Outputting some hardware information:

```
sudo lshw -class network
```

Interface names and descriptions:

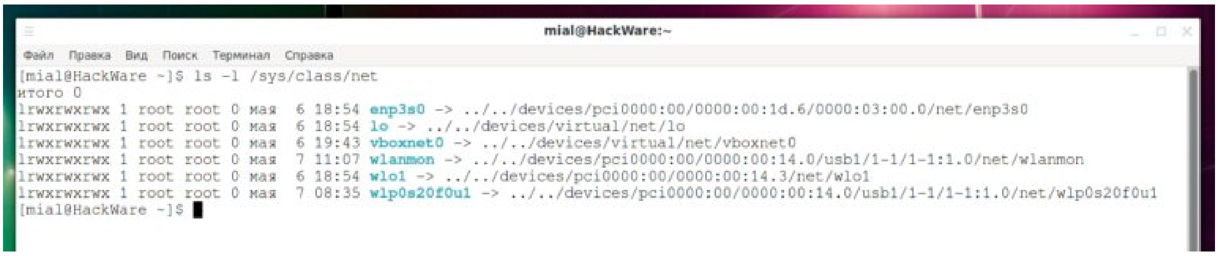
```
sudo lshw -class network -short
```

Output of everything that resembles network interfaces (along with virtual ones):

```
ip link show
```

The following way you can see the names of the interfaces and equipment:

```
ls -l /sys/class/net
```



This is how you can display the names of interfaces other than virtual ones:

```
find /sys/class/net -type l -not -lname '*virtual*' -printf '%f\n'
```

Please note that the interfaces created in this instruction are not considered virtual in this case, since they are tied to physical hardware.

Interface statistics can be viewed as follows:

```
cat /proc/net/dev
```

Additional sources

- About iw: https://wireless.wiki.kernel.org/en/users/documentation/iw

- Software access point: https://wiki.archlinux.org/title/Software_access_point
- https://ru.wikipedia.org/wiki/PHY
- https://www.kernel.org/doc/html/latest/driver-api/phy/phy.html
- https://www.kernel.org/doc/html/latest/networking/phy.html

Related articles:

- How to hack Wi-Fi (67.9%)
- How to create a Rogue Access Point (connected to the Internet through Tor) (66.2%)
- Fast and simple method to bypass Captive Portal (hotspot with authorization on the web-interface) (62.5%)
- How to Improve Wi-Fi Signal (61.9%)
- WiFi-autopwner 2: user manual and overview of new features (61.9%)
- bettercap 2.x: how to install and use in Kali Linux (RANDOM - 0.6%)

END Virtual Wi-Fi interfaces for simultaneous use of a Wi-Fi adapter in different modes